

USER GUIDE

1 – Installation

CNAppR is distributed as an R package and can be installed directly from GitHub using the devtools package:

```
install.packages("devtools")
devtools::install_github("Albertotorrejon/CNAppR")
library(CNAppR)
```

2 – Input data formats

CNAppR supports two entry points depending on the available data:

Workflow	Input	Key function
BAM pipeline	.bam file (WES or WGS)	run_bam_pipeline()
Pre-segmented	Segment table (.txt / .tsv)	read_cna_file()

2.1 – BAM pipeline input

A BAM file aligned to the human genome (hg19 or hg38) is required. The corresponding index file (.bai) must be present in the same directory. For whole-exome sequencing (WES), a BED file defining the capture kit target regions must also be provided. Supported sequencing platforms: Illumina (WES and WGS) and Oxford Nanopore (low-pass WGS and cfDNA).

[FIGURE: Pipeline diagram: BAM -> read_bam_qc() -> run_cbs_segmentation() -> calculate_cna_scores() -> FCS / BCS / GCS]

2.2 – Pre-segmented input

CNAppR accepts pre-segmented data from any segmentation algorithm (e.g. ASCAT, CNVkit, GATK4) as well as from SNP arrays (Illumina, Affymetrix) or aCGH platforms. The input must be a tab- or comma-separated file. File HEADER and COLUMN ORDER are mandatory.

Column	Type	Description
ID	character	Sample identifier
chr	integer	Chromosome (1-22); "chr" prefix is optional
loc.start	integer	Segment start position (bp)
loc.end	integer	Segment end position (bp)
seg.mean	numeric	Log2 copy-number ratio

Optional columns: purity (tumour purity, 0-1) and BAF (B-allele frequency). When provided, purity is used to correct copy-number amplitudes, and BAF enables detection of copy-number neutral loss-of-heterozygosity (CN-LOH) events.

[FIGURE: Example of minimum input file (MIF) -- same format as CNApp 1.0]

3 – Workflow 1 – BAM pipeline

The BAM pipeline processes sequencing files end-to-end without requiring external tools (no samtools, no FASTA reference). It consists of three sequential steps and can be run as a single call or step-by-step.

3.1 – One-call wrapper: run_bam_pipeline

WES (Illumina) example:

```
result <- run_bam_pipeline(
  bam_path      = "tumor.bam",
  sequencing_type = "illumina",
  experiment_type = "wes",
  targets_bed    = "capture_kit.bed",
  genome_build   = "hg19",
  gc_correction  = TRUE
)
result$scores    # FCS, BCS, GCS
result$segments  # segmented profile
```

WGS / Nanopore low-pass (cfDNA) example:

```
result <- run_bam_pipeline(
  bam_path      = "cfDNA_sample.bam",
  sequencing_type = "nanopore",
  experiment_type = "wgs",
  bin_size      = 1e6,
  genome_build   = "hg38",
  gc_correction  = TRUE
)
```

Coverage routing: when mean coverage is below 10×, CNAppR automatically routes to ichorCNA for segmentation and tumour fraction estimation (method = "auto", default). Force a specific method with method = "CBS" or method = "ichorCNA".

3.2 – Step-by-step pipeline

```
# Step 1 -- read BAM, count reads per bin, apply GC correction
qc <- read_bam_qc(
  "tumor.bam",
  sequencing_type = "illumina",
  experiment_type = "wes",
  targets_bed     = "capture_kit.bed"
)

# Step 2 -- CBS segmentation
seg <- run_cbs_segmentation(qc)

# Step 3 -- compute CNA scores
scores <- calculate_cna_scores(seg$segments)
```

3.3 – BAM pipeline parameters

Parameter	Default	Description
bam_path	--	Path to the indexed .bam file (.bai must exist alongside)
sequencing_type	"illumina"	"illumina" or "nanopore"
experiment_type	"wes"	"wes" (exome) or "wgs" (whole genome)
targets_bed	--	Capture kit BED file -- required for WES
genome_build	"hg38"	"hg19" or "hg38"

bin_size	500kb / 1Mb	500 kb for Illumina; 1 Mb recommended for low-coverage data
gc_correction	TRUE	Apply GC-content bias correction

3.4 – Panel of Normals (optional)

For tumour-only workflows, a Panel of Normals (PoN) can be built from matched normal BAM files to remove systematic technical biases:

```
pon <- build_pon(
  bam_paths      = c("normal1.bam", "normal2.bam", "normal3.bam"),
  sequencing_type = "illumina",
  experiment_type = "wes",
  targets_bed     = "capture_kit.bed"
)
result <- run_bam_pipeline("tumor.bam", pon = pon, ...)
```

4 – Workflow 2 – Pre-segmented input

For samples from SNP arrays, aCGH or any upstream segmentation tool. A demo dataset of 160 colorectal cancer samples (TCGA-COAD) is bundled with the package.

```
data <- read_cna_file(system.file("models", "demo.txt", package =
  "CNAppR"))

sample_ids <- unique(data$ID)
reseg_list <- lapply(sample_ids, function(id) resegment_sample(data,
  sample_id = id))
names(reseg_list) <- sample_ids

scores <- calculate_cna_scores(reseg_list)
```

4.1 – Re-segmentation parameters

The re-segmentation step corrects background noise and amplitude divergence due to technical variability, and re-adjusts copy-number thresholds using estimated purity (if provided).

Parameter	Default	Description
min_length	100,000 bp	Minimum segment length; shorter segments are discarded
dev_btw_segs	0.16	Max log2 difference between adjacent segments for merging
dev_tozero	0.16	Segments within this range of 0 are set to neutral
max_dist_segm	1,000,000 bp	Max distance between segments to consider merging
dev_baf	0.1	Max BAF deviation between segments (ignored if BAF absent)
Sample_type	"solid_tumor"	"solid_tumor" or "liquid_biopsy" – adjusts noise thresholds for cfDNA samples

4.2 – CNA callin thresholds (default values)

CNA level	Log2 ratio	Estimated copies
High-level gain	1.0	≥ 4 copies
Medium-level gain	0.58	[3 - 4) copies
Low-level gain	0.2	[2.3 - 3] copies
Low-level loss	-0.2	(1 - 1.7] copies
Medium-level loss	-1.0	(0.6 - 1] copies
High-level loss	-1.74	≤ 0.6 copies

The relation between seg.mean and estimated copy number:

$$\text{seg.mean} = \log_2(\text{number_of_copies} / 2)$$

4.3 – CNA classification

Each segment is classified based on its length relative to the chromosome:

Level	Criterion	Description
chromosomal	> 90% chromosome length	Whole-chromosome alteration
arm	> 50% arm length	Arm-level alteration (p or q)
focal	< arm threshold	Sub-arm alteration
diploid	No alteration	Segment within normal range

4.4 – Re-segmentation output

Re-segmented data can be exported for all samples as a data frame. Segments classified as diploid are included in the output but are NOT considered for computing CNA scores.

```
# Export re-segmented data
reseg_df <- do.call(rbind, reseg_list)
write.table(reseg_df, "resegmented_samples.tsv", sep="\t",
row.names=FALSE)
```

[FIGURE: Example re-segmentation plot for an individual sample (before / after)]

5 – CNA scores

Score	Description
FCS	Focal CNA Score -- count of sub-arm alterations weighted by size
BCS	Broad CNA Score -- count of arm- and chromosomal-level alterations
GCS	Global CNA Score -- normalised combination of FCS and BCS

```
scores <- calculate_cna_scores(reseg_list)
head(scores) # columns: FCS, BCS, GCS; rows: sample Ids
```

[FIGURE: Score distribution plots (density/boxplot) -- FCS, BCS and GCS]

6 – Visualisation

6.1 – Segmentation profile

Plot copy-number segments before and after re-segmentation for a single sample:

```
plot_segmentation(
  original_data      = data[data$ID == "sample_1", ],
  resegmented_data = reseg_list[["sample_1"]],
  sample_id         = "sample_1"
)
```

[FIGURE: Example segmentation plot -- before (top) and after (bottom) re-segmentation]

6.2 – Genome-wide CNA frequency

Plot genome-wide gain/loss frequencies across all samples. Use arm-level resolution for array/WGS data, or WES targets for exome cohorts:

```
# Arm-level (recommended for array and WGS data)

arm_ref <- system.file(
  "aux_files/segmented_files_hg19/autosomes_hg19_by_arms.txt",
  package = "CNAppR"
)

freq <- compute_region_frequencies(reseg_list, arm_ref,
                                  gain_thr = 0.23, loss_thr = -0.23)
plot_frequency_profile(freq, title = "Copy Number Frequency")

# WES targets (only when input data comes from WES)

wes_ref <- system.file("extdata/standard_exome_hg19.bed", package =
  "CNAppR")
freq_wes <- compute_region_frequencies(reseg_list, wes_ref,
                                       gain_thr = 0.23, loss_thr = -0.23)
plot_frequency_profile(freq_wes)
```

[FIGURE: Genome-wide frequency plot -- gains (red) and losses (blue) across chromosomes 1-22]

7 – Gene annotation

CNA segments can be annotated with the genes they overlap using Ensembl/UCSC reference data. This is particularly informative for focal alterations, where gene-level resolution is biologically meaningful.

```
annotated <- annotate_cna_genes(
  reseg_list[["sample_1"]],
  genome_build = "hg19"
)
```

8 – Clinical association

CNA scores can be statistically associated with clinical or molecular variables. Both parametric and non-parametric tests are applied according to variable type:

Variable type	Parametric	Non-parametric
Categorical (n = 2)	Student's t-test	Wilcoxon signed-rank test
Categorical (n > 2)	ANOVA	Kruskal-Wallis test
Numerical	Pearson correlation	Spearman correlation

```
results <- test_clinical_association(
  scores,
  clinical_data = clinical,
  method       = "both"
)
results$pval_parametric
results$pval_nonparametric
```

[FIGURE: Table of p-values -- rows: clinical variables, columns: FCS / BCS / GCS]

9 – Classification model

A Random Forest classifier can be trained to predict clinical or molecular subtypes from CNA scores. The user provides a labelled cohort and splits it into training and test sets:

```
install.packages("randomForest")

# Split 70 / 30
idx <- sample(nrow(scores), 0.7 * nrow(scores))
train_scores <- scores[idx, ]
test_scores <- scores[-idx, ]
train_labels <- labels[idx]      # named vector: sample_id → class
test_labels <- labels[-idx]

# Train
mod <- train_cna_classifier(train_scores, train_labels, ntree = 500)
mod$ooob_accuracy  # out-of-bag accuracy
mod$importance     # variable importance (FCS / BCS / GCS)

# Predict on test set
preds <- predict_cna_class(test_scores, mod$model)
head(preds)
#>   ID predicted_class ClassA ClassB
#>  s1 ClassA           0.82  0.18

# Save and reload the model
saveRDS(mod$model, "my_model.rds")
model <- load_prediction_model("my_model.rds")
```

[FIGURE: Classifier results -- accuracy, sensitivity and specificity by group (bar chart)]